You've implemented the logic in the `charge!` method of `Order`, so what goes in the newly created `ChargeOrderJob`? The purpose of job classes like `ChargeOrderJob` is to act as a glue between the controller—--which wants to run some logic later—--and the actual logic in the models.

Here's the code that implements this:

rails80/depot_qb/app/jobs/charge_order_job.rb

```
    class ChargeOrderJob < ApplicationJob
      queue_as :default

»     def perform(order, pay_type_params)
»       order.charge!(pay_type_params)
      end
    end
```

Next, you need to fire this job in the background from the controller.

Queuing a Background Job

Because background jobs run in parallel to the code in the controller, the code you write to initiate the background job isn't the same as calling a method. When you call a method, you expect that method's code to be executed while you wait. Background jobs are different. They often go to a queue, where they wait to be executed outside the controller. Thus, when we talk about executing code in a background job, we often use the phrase "queue the job."

To queue a job using Active Job, use the method `perform_later` on the job class and pass it the arguments you want to be given to the `perform` method you implemented above. Here's where to do that in the controller (note that this replaces the call to `OrderMailer`, since that's now part of the `charge!` method):

rails80/depot_qb/app/controllers/orders_controller.rb